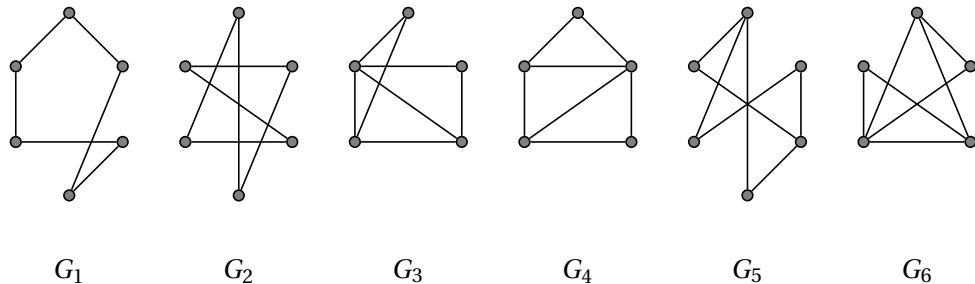


1 Généralités

- 1 Décider si les dessins suivants représentent les mêmes graphes.



□

- 2 Dessiner les graphes suivants :

1. Les sommets sont les faces d'un cube, deux sommets sont reliés si les faces correspondantes ont une arête du cube en commun.
2. Les sommets du graphe sont tous les sous ensembles à deux éléments de $\{1, 2, 3, 4\}$ deux sommets sont reliés si leur intersection est non vide.

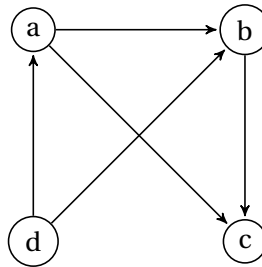
□

- 3 Reconstituer les graphes à partir des matrices d'adjacences suivantes :

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad B = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \end{pmatrix} \quad C = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{pmatrix}$$

□

- 4 On considère le graphe ci-dessous. Donnez sa matrice d'adjacence A . Calculer A^2 , A^3 et A^4 . Donner une interprétation des coefficients de ces matrices puis montrer cette relation par récurrence.



□

- 5 Cinq personnes se rencontrent tous les matins. Est-ce possible que chacune d'entre elles sert la main à uniquement trois personnes?

□

- 6 Montrez que dans un groupe de personnes, il y a toujours deux personnes ayant le même nombre d'amis présents.

□

- 7 Dans un tout petit pays, il n'y a que 15 villes. On peut aller de chaque ville à au moins 7 autres villes du pays par une autoroute. Peut-on se rendre, par autoroute, de la capitale du pays à chacune des autres villes?

□

2 Opérations de parcours dans un graphe

- 8 **Affichage - Parcours en largeur**
Écrire la procédure qui permet d'afficher les valeurs des sommets d'un graphe non orienté en suivant un parcours en largeur.

□

- 9 **Affichage - Parcours en profondeur préfixé**
Écrire la procédure qui permet d'afficher les valeurs des sommets d'un graphe orienté en suivant un parcours en profondeur préfixé.

□

- 10 **Connexe**
Écrire la fonction qui permet de vérifier si un graphe non orienté est connexe.

□

11

Chaîne - Parcours en largeur

On donne deux sommets s_1 et s_2 d'un graphe non orienté. Écrire la fonction qui permet de vérifier qu'il existe une chaîne reliant s_1 à s_2 . On vous demande d'utiliser le principe de parcours en largeur. □

12

Chemin - Parcours en profondeur

On donne deux sommets s_1 et s_2 d'un graphe orienté. Écrire la fonction qui permet de vérifier qu'il existe un chemin reliant s_1 à s_2 . On vous demande d'utiliser le principe de parcours en profondeur. □

13

Arbre enraciné

Écrire une fonction qui permet de vérifier si un graphe orienté est un arbre enraciné. □

14

Circuit

Dans un graphe orienté, on appelle circuit une suite d'arcs consécutifs (chemin) dont les deux sommets extrémités sont identiques.

On donne un sommet s d'un graphe orienté. Écrire la fonction qui permet de vérifier qu'il existe un circuit à partir de s . □

3 Ordre topologique

15

On considère la précondition suivante comme valide : le graphe est orienté, non valué, sans circuit et l'application de ce graphe non orienté est connexe.

Le tri topologique d'un graphe orienté acyclique $G = (S, A)$ consiste à ordonner linéairement tous ses sommets de telle sorte que si G contient un arc entre s_i et s_j , s_i apparaisse avant s_j .

Écrire l'algorithme permettant de déterminer l'ordre topologique dans un graphe. □

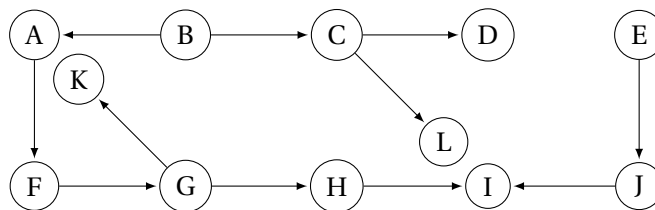


FIGURE 1 – Ordre topologique : E, J, B, C, D, L, A, F, G, K, H, I

Annexe A : Opérations sur les ensembles

Procédure **creerEnsemble** (ens: Ensemble de Élément (S))

Procédure **ajouter** (ens: Ensemble de Élément (E/S), s: Élément)

Procédure **supprimer** (ens: Ensemble de Élément (E/S), s: Élément)

Fonction **uneValeur** (ens: Ensemble de Élément) : Élément

Fonction **cardinalite** (ens: Ensemble de Élément) : entier

Fonction **existe** (ens: Ensemble de Élément, e: Élément) : booléen

Procédure **copier** (ens1: Ensemble de Élément, ens2: Ensemble de Élément (S))

Annexe B : Opérations sur les graphes

Procédure **creerGraphe** (g: Graphe de Élément (S), ORIENTE/NONORIENTE, VALUÉ/NONVALUÉ)

Procédure **ajouterSommet** (g: Graphe de Élément (E/S), s: Élément)

Procédure **supprimerSommet** (g: Graphe de Élément (E/S), s: Élément)

Procédure **ajouterArete** (g: Graphe de Élément (E/S), debut: Élément, fin: Élément, [val : Nombre])

Procédure **supprimerArete** (g: Graphe de Élément (E/S), debut: Élément, fin: Élément)

Procédure **modifierValeurArete** (g: Graphe de Élément (E/S), debut: Élément, fin: Élément, val : Nombre)

Fonction **existeArete** (g: Graphe de Élément, debut: Élément, fin: Élément) : booléen

Fonction **recupererArete** (g: Graphe de Élément, debut: Élément, fin: Élément) : Arete

Enregistrement Arete

debut: Élément

fin: Élément

[valeur : Nombre]

Fin Enregistrement

Procédure **ensSuivant** (g: Graphe de Élément, s: Élément, ens: Ensemble de Élément (S))

Procédure **ensPrecedent** (g: Graphe de Élément, s: Élément, ens: Ensemble de Élément (S))

Procédure **ensSommets** (g: Graphe de Élément, ensS: Ensemble de Élément (S))

Procédure **ensAretes** (g: Graphe de Élément, ensA: Ensemble de Arete (S))

Procédure **copier** (g1: Graphe de Élément, g2: Graphe de Élément (S))